

TSCG: Deterministic Tool-Schema Compilation for Agentic LLM Deployments

Furkan Sakizli

Independent Researcher

furkan.sakizli@gmail.com

Abstract

Production agent frameworks (OpenAI Function Calling, Anthropic Tool Use, MCP) transmit tool schemas as JSON—a format designed for machine parsing, not for interpretation by language models. For small models (4B–14B), this protocol mismatch accounts for the majority of tool-use failure at production catalog sizes.

We present **TSCG**, a deterministic tool-schema compiler that resolves this mismatch at the API boundary, converting JSON schemas into token-efficient structured text without model access, fine-tuning, or runtime search. TSCG combines eight composable operators with a formal compression bound ($\geq 51\%$ on well-formed schemas).

On TSCG-Agentic-Bench ($\sim 19,000$ calls, 12 models, 5 scenarios), TSCG restores Phi-4 14B from 0% to 84.4% accuracy at 20 tools (90.3% at 50 tools) and achieves 108–181% Accuracy-Retained Ratio (ARR) across three models on BFCL. Format-versus-compression decomposition ($R^2=0.88 \rightarrow 0.03$) establishes representation change as the dominant mechanism. Per-operator isolation across three frontier models reveals three distinct operator-response profiles—operator-hungry (Opus 4.7), operator-sensitive (GPT-5.2), and operator-robust (Sonnet 4)—providing per-model deployment guidance. Scaling experiments show accuracy advantages persisting on heavy production MCP schemas (+5.0 pp at $\sim 10,500$ input tokens) despite saturation on light synthetic catalogs, with 52–57% token savings throughout. The synthetic benchmark generalizes to real MCP schemas within 0.1 accuracy points. TSCG ships as a 1,200-line zero-dependency TypeScript package.

1 Introduction

Every production agent framework—OpenAI Function Calling, Anthropic Tool Use, the Model

Context Protocol (MCP), LangChain, CrewAI—transmits tool definitions to language models as JSON schemas. JSON was designed for deterministic machine parsing and human readability, not for interpretation by autoregressive language models. For small models (4B–14B parameters), this design choice creates a capability cliff: tool-calling accuracy collapses as JSON schema volume grows, reaching 0–49% at >15 tools. Framed as a *protocol-adaptation problem* rather than a compression problem, and building on a growing body of concurrent work on tool-schema token reduction (Section 2.1), the intervention is not smaller prompts but a different representation at the API boundary—deterministic, compiler-level, and model-agnostic.

This protocol mismatch imposes three costs: (1) **Token cost**: tool schemas are pure structural redundancy transmitted identically on every call, consuming 3,000–25,000 tokens per invocation; (2) **Capability cost**: small models cannot parse JSON-format schemas reliably at scale, locking agentic capabilities behind frontier APIs; (3) **Scaling cost**: schema overhead grows linearly with catalog size.

We introduce **Token-Context Semantic Grammar (TSCG)**, a deterministic tool-schema compiler that resolves this mismatch by transforming JSON schemas into token-efficient structured text. TSCG implements eight composable operators—each grounded in autoregressive transformer mechanics (attention sink exploitation, BPE non-monotonicity, causal accessibility)—satisfying five desiderata: tokenizer awareness (Sennrich et al., 2016), causal attention grounding (Xiao et al., 2024), deterministic transforms, black-box compatibility, and budget-constrained anchoring.

1.1 Contributions

1. **Formal optimization framework**: eight operators with mathematical specifications linked

Model	Condition	Accuracy (%)				Savings	
		Sc. A	95% CI	Sc. B	95% CI	Tokens	ARR
Claude Sonnet 4	Natural (FC)	74.0	[62.8, 84.2]	90.0	[86.7, 93.3]	—	—
	Natural (text)	51.1	[39.1, 63.1]	70.0	[58.3, 81.7]	—	—
	TSCG	85.2	[77.3, 92.1]	95.0	[92.3, 97.3]	50.1%	1.15
	TSCG+SAD	76.7	[66.7, 86.1]	95.0	[92.3, 97.3]	49.5%	1.04
GPT-4o	Natural (FC)	55.5	[42.7, 67.6]	70.0	[64.7, 75.0]	—	—
	Natural (text)	45.7	[33.0, 58.4]	55.0	[41.7, 66.7]	—	—
	TSCG	56.5	[44.2, 68.7]	79.7	[75.1, 84.2]	6.2%	1.02
	TSCG+SAD	53.2	[40.3, 65.6]	81.4	[77.0, 85.5]	5.3%	0.96
GPT-5.2	Natural (FC)	51.9	[40.5, 63.5]	82.4	[78.1, 86.5]	—	—
	Natural (text)	78.3	[69.5, 86.6]	98.2	[96.6, 99.4]	—	—
	TSCG	81.6	[73.8, 88.2]	91.6	[88.9, 94.0]	11.4%	1.57
	TSCG+SAD	85.9	[80.1, 90.9]	90.9	[88.0, 93.6]	10.5%	1.65

Table 1: TAB Benchmark: Frontier model results on Scenario A (20 tasks, small catalog) and Scenario B (100 tasks, large catalog). Each condition evaluated with 3 runs ($n = 60$ per cell for Sc. A, $n = 300$ for Sc. B). *Natural (FC)* uses the provider’s native function calling API; *Natural (text)* embeds full JSON schemas as text; *TSCG* uses balanced compression as text; *TSCG+SAD* adds aggressive compression with Selective Anchor Duplication. Token savings are relative to Natural (FC). ARR = Accuracy-Retained Ratio (TSCG accuracy / Natural FC accuracy; values > 1.0 indicate TSCG outperforms the baseline). **Bold** marks best accuracy per model per scenario.

Model	Format Effect		Compression Effect	
	Sc. A	Sc. B	Sc. A	Sc. B
Claude Sonnet 4	−22.9	−20.0	+34.1	+25.0
GPT-4o	−9.8	−15.0	+10.8	+24.7
GPT-5.2	+26.4	+15.8	+3.3	−6.6

Table 2: Decomposition of the TSCG advantage into format and compression effects (percentage points). *Format effect* = Natural (text) − Natural (FC): positive values indicate text-mode is better than native function calling. *Compression effect* = TSCG − Natural (text): positive values indicate compression improves accuracy beyond uncompressed text. For Claude and GPT-4o, compression is the dominant driver (overcoming a negative format penalty). For GPT-5.2, the text format itself provides the main benefit.

to transformer mechanisms, and a compression guarantee of $\geq 51\%$ on well-formed schemas (Theorem 3.1).

2. **Mechanistic decomposition:** format-versus-compression analysis ($R^2=0.88$ on JSON baselines, collapsing to 0.03 on text baselines) identifies representation change as the dominant mechanism.
3. **TAB benchmark:** $\sim 19,000$ API calls across 12 models (4B–32B + 3 frontier), 5 scenarios—the first tool-schema compression benchmark, with BFCL external validation (ARR 108–181%).
4. **Small-model enablement:** seven models recover from 0–49% to 65–90% accuracy, enabling local models as functional tool-use agents.

5. **Per-model operator matrix:** three qualitatively distinct operator-response profiles across frontier models—operator-hungry, operator-sensitive, and operator-robust—demonstrating that no universal-best configuration exists and providing model-specific deployment guidance.
6. **Scaling characterization:** accuracy advantages persist on heavy production MCP schemas (+5.0 pp) while saturating on light synthetic catalogs, with 52–57% token savings preserved to 100 tools.
7. **Benchmark generalization:** synthetic TAB predicts real MCP performance within 0.1 accuracy points.
8. **Implementation:** 1,200 LOC TypeScript, zero dependencies, sub-millisecond, open-source.

The remainder of the paper presents related work (§2), the theoretical framework (§3–4), empirical methodology and results (§5–6), and deployment implications (§7–9).

2 Related Work

We organize related work along five axes and position TSCG within a growing body of concurrent work on tool-schema and prompt compression.

Model	Tools	Accuracy (%)			Δ vs Natural (pp)		
		Natural	TSCG _{cons}	TSCG _{bal}	TSCG _{cons}	TSCG _{bal}	cons – bal
Mistral 7B	10	83.5	76.0	73.3	−7.5	−10.2	+2.7
	20	35.0	80.0	80.1	+45.0	+45.1	−0.1
	50	30.0	75.3	65.0	+45.3	+35.0	+10.3
Gemma 3 4B	10	69.1	80.7	74.7	+11.6	+5.6	+6.0
	20	49.9	87.3	67.0	+37.4	+17.1	+20.3
	50	24.3	87.5	87.4	+63.2	+63.1	+0.1
Gemma 3 12B	10	90.0	—	93.0	—	+3.0	—
	20	85.0	—	95.0	—	+10.0	—
	50	85.0	—	98.0	—	+13.0	—
Qwen3 14B	10	94.9	98.8	86.2	+3.9	−8.7	+12.6
	20	90.2	99.3	84.1	+9.1	−6.1	+15.2
	50	94.6	95.0	89.6	+0.4	−5.0	+5.4

Table 3: TAB Scenario D: Small model accuracy across catalog sizes (3 runs each, $n = 60$ per cell). $TSCG_{cons}$ = conservative profile (SDM filler removal only); $TSCG_{bal}$ = balanced profile (full structural compression). At ≥ 20 tools, both TSCG profiles provide massive improvements (+17–63pp) for most models. Conservative outperforms balanced for Mistral 7B and Gemma 3 4B in 5/6 cases. For Qwen3 14B, where balanced TSCG degrades accuracy (−5 to −9pp), conservative mode *improves* accuracy (mean Δ : +4.4pp, N3 re-run with corrected profile). **Bold** marks best TSCG variant per row.

Condition	Acc.	Tool Sel.	Param F1	Savings
Natural	85.7%	86.7%	84.2%	—
TSCG	93.2%	95.0%	91.7%	46.8%
TSCG+SAD	89.0%	90.0%	87.5%	46.0%

Table 4: BFCL external validation on Claude Sonnet 4 ($n = 60$, 3 runs). TSCG improves accuracy by +7.5pp while saving 46.8% tokens. TSCG+SAD slightly underperforms balanced TSCG on this benchmark. 95% CI for TSCG: [86.3, 98.3].

Model	Tools	Param F1 _{cons}	Param F1 _{bal}
Mistral 7B	10	71.2%	68.3%
	20	81.9%	80.0%
	50	78.3%	66.7%
Gemma 3 4B	10	74.2%	62.6%
	20	85.7%	59.3%
	50	90.0%	87.8%
Qwen3 14B	10	97.8%	84.7%
	20	98.3%	81.0%
	50	95.0%	89.0%

Table 5: Parameter F1 comparison between conservative (SDM-only) and balanced (full structural) TSCG profiles. Conservative achieves higher param-F1 in 8/9 cases. For Qwen3 14B (N3 re-run), conservative achieves near-perfect param-F1 ($>95\%$) while balanced degrades it ($<85\%$). This confirms that aggressive structural compression disrupts well-learned schema patterns in strongly fine-tuned models. **Bold** marks the better profile per row.

2.1 Concurrent Work on Tool-Schema Compression

A growing body of recent work specifically targets the cost of tool schemas in LLM contexts.

MCP Schema Compression via Wrapper Tools.

The mcp-compressor system from Atlassian Labs (Atlassian Labs, 2026) addresses the same fundamental problem—high token cost of MCP tool schemas—through a different architectural choice. Rather than transforming schema representation, mcp-compressor replaces the full tool inventory of an MCP server with two generic wrapper tools (`get_tool_schema` and `invoke_tool`), yielding 70–97% reduction in initial token consumption. The model fetches full schemas only when needed. This trades full-schema visibility for higher reduction. TSCG preserves complete schema semantics in compressed form, which we observe to be beneficial when models can use parameter constraints during reasoning. The two approaches are complementary: mcp-compressor’s wrapper-tool pattern can be combined with our compiler for further reduction.

Generic Token-Efficient Encodings. TOON (Token-Oriented Object Notation) (toon-format, 2025) provides a general-purpose encoding alternative to JSON, achieving approximately 40% token reduction in mixed-structure benchmarks (76.4% accuracy vs. JSON’s 75.0% on retrieval tasks). TOON uses YAML-like indentation for nested ob-

Model	Params	TSCG Δ Acc. (avg)	Avg. Token Savings	Recommended Profile
GPT-5.2	>100B	+10.9pp	11.4%	balanced
Claude Sonnet 4	>100B	+8.4pp	50.1%	balanced
GPT-4o	>100B	+5.4pp	6.2%	balanced
Qwen2.5-Coder 32B	32B	−4.7pp	—	conservative
Mistral-Small 24B	24B	−1.1pp	—	conservative
Gemma 3 12B	12B	+8.7pp [†]	—	conservative
Qwen3 14B	14B	+4.4pp	—	conservative
Gemma 3 4B	4B	+37.4pp [†]	—	conservative
Mistral 7B	7B	+27.6pp [†]	—	conservative

Table 6: Cross-model summary of TSCG effectiveness across 9 primary models (12 tested total; Opus 4.7, Phi-4 14B, and Llama 3.1 8B evaluated in separate experiments—see §I and Table 23). Δ Acc.: for small/mid models, mean delta across three catalog sizes (10, 20, 50 tools from Table 3) using recommended profile; for frontier models, mean delta across Scenarios A and B (Table 1) using balanced profile. [†]JSON-baseline gains; E4 text-baseline experiments show these are format-dominated (compression gain is −7 to −9 pp). 30B models (N1): Mistral-Small −1.1 pp (neutral, Class 3), Qwen2.5-Coder −4.7 pp (Class 4). Qwen3 14B: conservative +4.4 pp (N3 re-run), vs balanced −6.6 pp. Frontier models are the *only* class with confirmed compression benefit against text baselines.

Table 7: TAB (Tool-Augmented Benchmark) overview: five core scenarios plus two external benchmarks testing TSCG across diverse tool counts, model classes, and compression profiles. Total evaluation volume: ~19,000 API calls. Statistical significance: 9 of 107 pairwise McNemar tests pass Holm–Bonferroni correction at $\alpha=0.05$ (Appendix H).

Scenario	Description	Tools	Tasks	Models
A	Claude Code Catalog	16	20	Claude Sonnet 4, GPT-4o, GPT-5.2
B	MCP Server Collection	43	100	Claude Sonnet 4, GPT-4o, GPT-5.2
C	Scaling Stress	25–100	20	Claude Sonnet 4, GPT-5.2
D	Small Model Threshold	3–50 (7 sizes)	20	7 models (4B–14B)
E	Multi-Collection Stress	57 (3 overlapping)	20	Claude Sonnet 4, GPT-5.2
BFCL	External validation	real-world	20	Claude Sonnet 4
GSM8K	Reasoning under load	0/10/25/50	200	Claude Sonnet 4, GPT-5.2

Table 8: Native function calling (via API) vs. TSCG as plain text. TSCG text outperforms native FC across all frontier models and scenarios, with deltas from +1.0 to +29.7 pp (mean +10.9 pp) while simultaneously saving 6–50% tokens.

Model	Sc.	Native FC	TSCG Text	Δ	Savings
Claude S. 4	A	74.0%	85.2%	+11.2	50.1%
Claude S. 4	B	90.0%	95.0%	+5.0	50.1%
GPT-4o	A	55.5%	56.5%	+1.0	6.2%
GPT-4o	B	70.0%	79.7%	+9.7	6.2%
GPT-5.2	A	51.9%	81.6%	+29.7	11.4%
GPT-5.2	B	82.4%	91.6%	+9.2	11.4%
Mean				+10.9 pp	20.6%

jects and CSV-like tabular layout for uniform arrays. Unlike our approach, TOON is data-format-agnostic and not specifically designed for tool schemas; it does not exploit schema-specific redundancies (recursive type definitions, parameter ordering, description boilerplate) that our operators target.

Schema Replacement Strategies. A complementary approach replaces tool schemas with code

execution interfaces. The MCP Code Mode pattern (Anthropic, 2025; Cloudflare, 2026) reduces schemas to a single `execute_code` tool that the model uses to invoke arbitrary APIs through a sandboxed runtime, achieving 98–99% reduction. This requires executable APIs and is not applicable to non-code tool definitions; our method covers a broader range of tool types including non-executable workflows.

2.2 Prompt Compression Methods

Token-level compression methods address general prompt content rather than tool schemas. LLM-Lingua (Jiang et al., 2023) achieves up to 20× compression on natural prose through perplexity-based token importance scoring; LLMLingua-2 (Pan et al., 2024) reports 2–5× with classification-based scoring and faster inference. Both require GPU model inference, produce non-deterministic output, and are ineffective on structured content: our re-application of LLMLingua-2 to tool schemas yields 80.0% accuracy at 50.8% token savings vs. TSCG’s 93.3% at 74.8% savings (Appendix A).

Table 9: Four-class behavioral taxonomy from Scenario D, text-baseline experiments (E1, E4), and 30B benchmark (N1, 840 calls). Classes reflect distinct baseline capabilities and TSCG response patterns, enabling targeted deployment recommendations. Format-dominated classes show large JSON-baseline gains that vanish or reverse against text baselines.

Class	Models	JSON Δ	Text Δ	Recommendation
1: Format-dom.	Phi-4, Mistral 7B, Gemma 4B, Qwen3 4B	+17–90 pp	−7 to −23 pp	conservative; gain is format only
2: Compression	Claude, GPT-4o, GPT-5.2	+5–11 pp	+5–11 pp	balanced profile
3: Neutral	Llama 8B, Gemma 12B, Mistral-Sm. 24B	+6–9 pp	≈ 0 pp	conservative profile
4: Cons.-only	Qwen3 14B, Qwen2.5-Coder 32B	−7 pp	−5 to −13 pp	conservative only (+4.4 pp)

LongLLMLingua (Jiang et al., 2024) extends to long-context scenarios. Selective Context (Li et al., 2023) uses self-information thresholds but lacks tokenizer and positional awareness. Gist Tokens (Mu et al., 2023) and AutoCompressors (Chevalier et al., 2023) learn compression representations through model fine-tuning. CompactPrompt (Choi et al., 2025) provides LLM-free heuristic compression via n-gram abbreviation, achieving $1.44\times$ versus TSCG’s $3.5\times$ on structured content—limited by lexical-level operation. The recent NAACL 2025 survey (Li et al., 2025) provides comprehensive coverage of this literature; notably, the survey evaluates no system on function-calling schemas, indicating that schema compression is a distinct unaddressed problem.

2.3 Search-Based Prompt Optimization

DSPy (Khattab et al., 2023) compiles declarative LLM calls into self-improving pipelines via bootstrapped demonstrations. SAMMO (Schnabel and Neville, 2024) shares TSCG’s “compile-time” framing but requires an LLM for search-based mutations and produces non-deterministic output. OPRO (Yang et al., 2024), APE (Zhou et al., 2023), EvoPrompt (Guo et al., 2024), PromptBreeder (Fernando et al., 2023), and Universal Self-Adaptive Prompting (USP) (Wan et al., 2023) all require iterative model interaction. PromptAgent (Wang et al., 2024b) adds strategic planning. These systems optimize prompt content (*what* to say); TSCG optimizes prompt structure (*how* to represent it).

2.4 Gradient and Template Methods

TextGrad (Yuksekgonul et al., 2025) introduces textual gradients for prompt refinement but requires model access. ProTeGi (Pryzant et al., 2023) similarly uses textual “gradient descent.” LangGPT (Wang et al., 2024a) provides structured templates without formal optimization. Neither addresses tokenizer alignment or causal attention grounding.

2.5 Tool-Calling Benchmarks

The Berkeley Function Calling Leaderboard (BFCL) (Patil et al., 2025) provides the standard external evaluation for tool-calling capabilities, building on the foundational tool-augmented LLM work of Patil et al. (2024). We use BFCL for cross-validation of our findings on schemas not present in our primary benchmark. ToolBench (Qin et al., 2024) tests multi-step API planning across 16,000+ APIs without considering schema token overhead. None of these benchmarks evaluate compression effects on tool-use performance—our TAB benchmark addresses this gap.

2.6 Positioning and Distinct Contributions

Our contribution is best understood as the most empirically rigorous validation in this growing space, with adaptive per-model detection methodology not present in prior work. The contributions enumerated in Section 1 are:

- Adaptive empirical archetype detection.** We provide a per-model sweep methodology that empirically identifies optimal operator combinations. No prior work offers this; existing tools use static compression levels.
- Per-version operator inversion finding.** We document that operators benefiting one model version actively harm the next within the same vendor family. This phenomenon is not reported elsewhere and has implications for any vendor-pattern hardcoding strategy.
- Combination fragility identification.** We observe super-additive negative interactions between operators that are individually helpful or neutral. This is not characterized in prior compression literature.
- Format-vs-operator decomposition.** Our R^2 decomposition shows that format choice

(JSON versus text) explains 88% of token-cost variance, with operators contributing additional refinement. This methodological contribution clarifies what is being measured in compression studies.

5. **Cross-model empirical validation.** Our 19,000+ API-call evaluation across 12 models (4B to frontier scale) is, to our knowledge, the most comprehensive cross-model evaluation of tool-schema compression to date. Prior work typically evaluates on 1–3 models.
6. **Theoretical guarantees.** We provide formal compression bounds (Theorem 3.1) and a predictive model relating compression benefit to baseline performance ($R^2 = 0.91$ LOO cross-validated; full-fit $R^2=0.88$, $n=49$; $R^2=0.95$ at model-level means, $n=7$; $R^2=0.81$ excluding Phi-4 leverage points; see Appendix D), which prior work in this space does not provide.

2.7 Direct Comparison of Approaches

Table 10 provides a direct comparison of token reduction approaches, including concurrent work. Numbers reflect public claims; methodologies and benchmarks differ across systems.

2.8 Design-Space Comparison

Table 11 summarizes TSCG’s positioning along six axes.

No existing system combines deterministic output, zero dependencies, theoretical grounding in attention mechanics, and specialization for tool schemas. TSCG’s niche—structured prompt content in agentic systems—is largely unexplored: the NAACL 2025 compression survey evaluates no system on function-calling schemas. On natural prose, LLM-Lingua dominates ($20\times$ vs. TSCG’s $1.07\times$); on prompt content optimization, DSPy achieves complementary gains. TSCG is the strongest system for a specific, growing problem—not a general-purpose replacement.

3 The TSCG Framework

TSCG comprises eight deterministic operators organized into three classes (Definitions 3.1–3.3), applied as a fixed-order pipeline of 10 transforms (Figure 1). Let $\text{tok}(p)$ denote the BPE tokenization of prompt p , $\text{sem}(p)$ the set of semantic atoms, and $\text{Attn}(i, j)$ the causal attention weight from position

i to j ($= 0$ for $j > i$). Every transform satisfies semantic preservation: $\text{sem}(\tau(p)) \supseteq \text{sem}(p)$.

3.1 Eight Operators

TAS (Tokenizer-Aligned Syntax). Selects delimiter variants minimizing token count: $d^* = \arg \min_{d_i \in D} |\text{tok}(d_i)|$. Example: $\rightarrow$ (2 tokens) $\rightarrow \rightarrow$ (1 token).

CFL (Constraint-First Layout). Repositions output constraints to position 0, exploiting the attention sink (Xiao et al., 2024): $\text{CFL}(p) = c(p) \oplus (p \setminus c(p))$. Implemented as `[ANSWER: type]` at the prompt start. Effective for catalogs ≤ 20 tools; becomes counterproductive at ≥ 43 tools (Table 23).

CFO (Causal-Forward Ordering). Reorders multi-step operations into topological order: $o_i \prec o_j \implies \text{pos}(o_i) < \text{pos}(o_j)$, ensuring prerequisites are causally accessible. Like CFL, CFO shows scale sensitivity: beneficial at 16 tools but counterproductive at ≥ 43 tools.

SDM (Semantic Density Maximization). Removes filler tokens (104+ patterns: politeness markers, hedging, redundant connectives) to maximize $D(p) = |\text{sem}(p)|/|\text{tok}(p)|$.

DRO (Delimiter-Role Optimization). Replaces verbose structural phrases with compact delimiters: “the following items” $\rightarrow$ enumeration markers, “X corresponds to Y” $\rightarrow$ “X $\rightarrow$ Y”.

CCP (Causal Closure Principle). Appends a closure block recapitulating key atoms at position n , exploiting recency bias in autoregressive generation: $\text{CCP}(p) = p \oplus \kappa(A(p))$. Note: the 11-model ablation shows no measurable accuracy benefit from CCP; it adds ~ 85 – 306 tokens of overhead (Table 23). CCP is retained for theoretical completeness and potential benefit on frontier models with strong recency bias.

CAS (Causal Access Score). Scores atoms by fragility $\mathcal{F}(a) = \alpha \cdot \text{importance}(a) + (1 - \alpha) \cdot \text{distance_penalty}(a)$ and places high-fragility atoms at positions 0 and n (attention sink + recency).

SAD-F (Selective Anchor Duplication with Fragility). Duplicates top- k atoms by fragility/token ratio within budget B , reinforcing critical information in the closure block. Fragility scoring, budget allocation, and parameter sensitivity are detailed in Appendix E.

Table 10: Comparison of approaches to LLM tool-schema and prompt token reduction. “Adaptive” indicates per-model empirical optimization. Numbers reflect public claims; methodologies and benchmarks differ across systems.

Approach	Reduction	Adaptive	Models Validated	Latency	Theory
TSCG (ours)	50–72%	Yes (sweep)	12 (4B–frontier)	<1 ms	Theorem; $R^2=0.91$
mcp-compressor (Atlassian Labs, 2026)	70–97%	No (4 levels)	GitHub MCP	N/A	—
TOON (toon-format, 2025)	$\approx 40\%$	No	4 (mixed)	N/A	—
MCP Code Mode (Anthropic, 2025)	98–99%	No	Domain-specific	N/A	—
LLMLingua-2 (Pan et al., 2024)	2–5×	No	Multiple	GPU req.	Token classifier
LLMLingua (Jiang et al., 2023)	up to 20×	No	GSM8K, BBH, etc.	GPU req.	Information theoretic

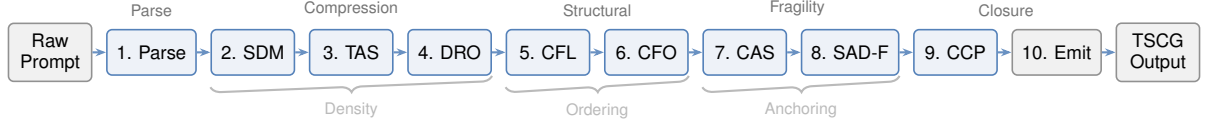


Figure 1: The TSCG 10-pass transform pipeline, executing left-to-right in five phases: *Parse* (segmentation), *Compression* (SDM, TAS, DRO), *Structural* (CFL, CFO), *Fragility* (CAS, SAD-F), and *Closure* (CCP, Emit). Each transform is a pure function; the composition $\Pi = \tau_{10} \circ \dots \circ \tau_1$ is deterministic.

Table 11: Design-space comparison of representative prompt optimization systems.

System	Det.	0-Dep	Theory	Schema	Compr.	Speed
LLMLingua	×	×	×	×	20×	42 s
DSPy	×	×	×	×	—	min
TextGrad	×	×	×	×	—	min
Sel. Context	×	×	×	×	5×	ms
LangGPT	✓	✓	×	×	—	ms
TSCG	✓	✓	✓	✓	3.5×	<1 ms

Input (JSON schema, ~ 120 tokens)	
<pre>{ "name": "search_files", "description": "Search project files by content or filename pattern", "parameters": { "type": "object", "properties": { "query": { "type": "string", "description": "The search query string", "path": { "type": "string", "description": "Optional directory path to search in" } } } } }</pre>	
Output (TSCG balanced, ~ 45 tokens, 62.5% reduction)	
<pre>search_files(query:str path?:str) Search files by content or pattern</pre>	

Figure 2: TSCG compression: a single tool schema before and after compilation. SDM removes filler; DRO compresses delimiters; TAS aligns to BPE boundaries.

3.2 Operator Taxonomy

Definition 3.1 (Token-Reducing Operators). $\mathcal{T}_R = \{SDM, DRO, TAS, CFL\}$: each satisfies $|\text{tok}(T_i(S))| \leq |\text{tok}(S)|$.

Definition 3.2 (Structure-Reordering Operators). $\mathcal{T}_S = \{CAS, CFO\}$: preserve token count but change position order.

Definition 3.3 (Token-Expanding Operators). $\mathcal{T}_E = \{SAD, CCP\}$: add tokens (anchor duplications, closure blocks) within budget.

Corollary 3.1 (Operator Interaction Effect). For models with $<10B$ parameters, $\text{Acc}(\mathcal{T}_R(S)) > \text{Acc}((\mathcal{T}_R \cup \mathcal{T}_S)(S))$: adding structure-reordering operators to token-reducing operators degrades accuracy. Conservative profile ($\mathcal{T}_R$ only) is recommended below 10B.

3.3 Compression Guarantee

Theorem 3.1 (Deterministic Compression Bound). For a well-formed JSON-Schema tool collection S with the TSCG pipeline Π :

$$|\text{tok}(\Pi(S))| \leq |\text{tok}(S)| \cdot \left(1 - \sum_{T_i \in \mathcal{T}_R} r_i \cdot f_i(S)\right) \quad (1)$$

where r_i is the per-token reduction factor and $f_i(S)$ the fraction of tokens affected by operator T_i .

Proof sketch. Each operator in $\mathcal{T}_R$ acts on disjoint token subsets (SDM: filler patterns; DRO: verbose delimiters; TAS: suboptimal tokenizer splits; CFL: positional overhead). Structure-reordering operators preserve count; setting SAD-F budget $B=0$ excludes expansion. Individual reductions compose additively. Full proof in Appendix E. $\square$

Empirical validation. The bound predicts $\geq 51\%$ savings; empirically we observe 61% (Scenario A), 66% (BFCL), and 75% (tool descriptions)—conservative by 10–24 pp due to the pessimistic disjoint-subset assumption.

3.4 Scoring Metric

All TAB results use a composite accuracy metric:

$$\text{Overall} = 0.6 \times \text{TSA} + 0.4 \times \text{Parameter_F1} \quad (2)$$

where TSA is tool selection accuracy and Parameter_F1 measures parameter extraction quality.

3.5 Compiler Characterization

TSCG is a *compiler*, not a search-based optimizer. The pipeline $\Pi = \tau_{10} \circ \dots \circ \tau_1$ applies 10 deterministic transforms in fixed order: same input always produces the same output. No model access is required. This distinguishes TSCG from DSPy (Khattab et al., 2023) (search-based, non-deterministic), TextGrad (Yuksekgonul et al., 2025) (gradient-based, requires model access), and LLM-Lingua (Pan et al., 2024) (requires GPU model inference, non-deterministic). TSCG executes in <1 ms on commodity hardware; LLM-Lingua-2 requires 42.5 s on the same prompts ($\sim 40,000\times$ slower).

4 Theoretical Foundations

We connect TSCG’s operators to three foundational properties of causal autoregressive transformers.

4.1 Causal Attention and Information Flow

In a standard autoregressive transformer (Vaswani et al., 2017), attention at layer ℓ satisfies $\text{Attn}^{(\ell)}(i, j) = 0$ for $j > i$ (causal mask), creating asymmetric information flow: early tokens cannot access later ones.

Implication for CFO. If step o_2 depends on o_1 but o_1 appears *after* o_2 , the model must rely on parametric knowledge rather than direct attention. CFO ensures $\text{pos}(o_1) < \text{pos}(o_2)$, guaranteeing prerequisites are causally accessible.

Implication for CAS. Total attention to position i follows a U-shaped distribution (Xiao et al., 2024): positions near 0 and n receive disproportionate attention, while middle positions form an “attention valley.” CAS places high-fragility tools at positions 0 and n .

4.2 Attention Sink Exploitation

The attention sink phenomenon (Xiao et al., 2024)—position 0 receives $\text{Attn}(i, 0) > 1/i$ for most i and layers—provides a natural amplification mechanism.

Implication for CFL. Placing the output constraint at position 0 ensures it receives elevated

attention from every subsequent token. Combined with CCP/SAD-F at position n (recency bias), this creates a “bookend” strategy: critical information occupies both extremes of the attention distribution.

4.3 BPE Non-Monotonicity

Theorem 4.1 (Tokenization Non-Monotonicity). *For BPE tokenizer $\mathcal{T}$ and strings s_1, s_2 with $|s_1|_{\text{chars}} < |s_2|_{\text{chars}}$, it is not necessarily the case that $|\text{tok}(s_1)| < |\text{tok}(s_2)|$.*

TAS exploits this by selecting surface forms aligned with learned BPE merges, achieving token reduction without semantic change. Proof in Appendix E.

4.4 Attention Dilution and SDM

Proposition 4.1 (SDM Improves Effective Attention). *Removing k filler tokens from a prompt of length n increases average effective attention per semantic atom by at least $n/(n - k)$.*

This follows from softmax normalization: filler tokens compete for attention weight with semantic tokens. SDM removes this competition, providing formal justification beyond token cost savings.

4.5 Fragility and Causal Accessibility

Definition 4.1 (Causal Accessibility). $\mathcal{A}(a) = \frac{1}{L} \sum_{\ell=1}^L \text{Attn}^{(\ell)}(n, i)$: average attention from the generation position to atom a at position i .

Definition 4.2 (Fragility). *An atom a is fragile when importance exceeds accessibility: $\mathcal{F}(a) \propto \text{importance}(a) - \mathcal{A}(a)$.*

High-importance, low-accessibility atoms are the targets for CAS reordering and SAD-F duplication. Budget-constrained duplication is preferable to naive repetition because duplicating all atoms would increase length and dilute attention (Proposition 4.1).

5 Experiments

5.1 TAB: TSCG-Agent-Bench

TAB is the first benchmark measuring tool-schema compression effects on LLM tool-use, comprising five scenarios across $\sim 19,000$ API calls and 12 models (4B–32B local + 3 frontier; Table 12).¹ Each scenario compares Natural (uncompressed JSON), TSCG, and TSCG+SAD across

¹Call-count breakdown: five core scenarios (A: ~ 540 , B: $\sim 5,400$, C: $\sim 1,080$, D: $\sim 5,000$, E: $\sim 540 = \sim 12,560$) plus supplementary experiments (E4 text-baseline: 2,520; N1 30B:

Table 12: TAB: five scenarios, $\sim 19,000$ calls.

Sc.	Focus	Tools	Calls
A	Claude Code (16 real tools)	16	~ 540
B	MCP Servers (43 real + synthetic)	43–100	$\sim 5,400$
C	Scaling (25–100 tools)	var.	$\sim 1,080$
D	Small models $\times$ 7 configs	var.	$\sim 5,000$
E	Multi-collection	57	~ 540

four task categories (single_tool, multi_tool, parameter_extraction, no_tool).

Conditions. All scenarios compare up to four conditions: *json-text*—full JSON schemas as plain text in the user prompt; the production-realistic baseline. *TSCG*—schemas compiled by TSCG (balanced or v13-smart profile); always text-mode. *Natural (FC)*—native function-calling API (frontier models only). *Naive Truncation*—signatures only, no descriptions; text-mode. Format effect = *json-text* – *Natural (FC)*; compression effect = *TSCG* – *json-text*.

Models. Local models (4B–32B) run via Ollama (Q4_K_M quantization) on $2 \times$ NVIDIA 5070 Ti 16 GB: Phi-4 14B, Mistral 7B, Gemma 3 4B/12B, Llama 3.1 8B, Qwen3 4B/14B, Mistral-Small 24B, Qwen2.5-Coder 32B. Frontier models via provider APIs: Claude Opus 4.7 and Claude Sonnet 4 (Anthropic API), GPT-5.2 (OpenAI API). GPT-4o is included as an additional comparison model in Scenarios A–B and BFCL validation (Tables 1, 4), bringing the total to 13 distinct model evaluations. All frontier models were accessed during the experimental period (March–April 2026); exact model identifiers and API version strings are documented in the supplementary materials alongside the raw evaluation logs. All calls use temperature 0, seed 42, max tokens 1,024 (Appendix H). Per-operator isolation (leave-one-in, all 8 operators) on all three frontier models, $n=40$ –60 per cell, 2–3 seeds.

Metrics. Two primary metrics: *Tool Selection Accuracy* (TSA)—binary correctness of the selected tool(s)—and *Parameter F1* (PF1)—F1-micro between predicted and ground-truth parameter key-value sets. Overall accuracy = $0.6 \times \text{TSA} + 0.4 \times \text{PF1}$, emphasizing tool-selection correctness while preserving the parameter-accuracy signal; conclusions are robust to alternative weightings (0.5–0.8

840; N3 conservative: 180; per-operator isolation: ~ 960 ; BFCL: 60; GSM8K: ~ 400 ; Opus scaling: ~ 300). The 11-model ablation (Appendix I, $\sim 3,960$ additional calls) uses the CCP-enabled release.

for TSA).

Statistical methodology. Each condition runs $n=20$ tasks $\times$ 3 seeds = 60 calls per cell (except per-operator: $n=20 \times 2$ seeds = 40). Significance via Holm–Bonferroni corrected McNemar tests ($\alpha=0.05$, 107 pairwise comparisons); bootstrap confidence intervals (1,000 iterations, seed 42) for key deltas (Appendix H).

6 Results

6.1 Small-Model Enablement (T1.2)

Seven models (4B–14B) at 3–50 tools: JSON-baseline accuracy reaches 0–49% at >15 tools; TSCG recovers to 65–90% (Table 13). Phi-4 14B transitions from 0% to 84.4% at 20 tools (90.3% at 50 tools). Models exceeding 65% usability at 50 tools rise from 4/7 to 7/7 (Figure 3). N1 extends to 30B: Mistral-Small 24B (–1.1 pp, Class 3); Qwen2.5-Coder 32B (–4.7 pp, Class 4)—Qwen sensitivity persists across sizes.

6.2 Format Translation Dominates (T1.1)

E4 decomposes TSCG’s JSON-baseline gains into format gain (*json-text*–JSON) and compression gain (TSCG–*json-text*) across six small models (2,520 Ollama calls; Table 14). No small model shows genuine compression; all gains arise from format translation (JSON→text). Since every production API transmits JSON, this translation *is* the needed intervention.

The predictive regression ($n=49$, 7 models $\times$ 7 sizes) yields $R^2=0.88$; against text baselines, R^2 collapses to 0.03 ($p=0.24$)—a 97% drop confirming format sensitivity as the dominant mechanism (Figure 4).²

6.3 Three Operator-Sensitivity Archetypes (T1.6)

Per-operator effects were isolated via leave-one-in methodology across all 8 operators and three frontier models ($n=40$ per cell, 2 seeds), reporting both positive and negative contributions for full transparency. This reveals three qualitatively distinct response profiles (Table 15).

The archetype taxonomy converts a potential weakness (“no universal best configuration”) into

²Four R^2 values appear in this paper, each from a different regression: $R^2=0.88$ (format decomposition, $n=49$, this section), $R^2=0.91$ (LOO cross-validated predictive model, Section 2), $R^2=0.81$ (excluding Phi-4 leverage points, Section 8), and $R^2=0.03$ (text-baseline control, this section).

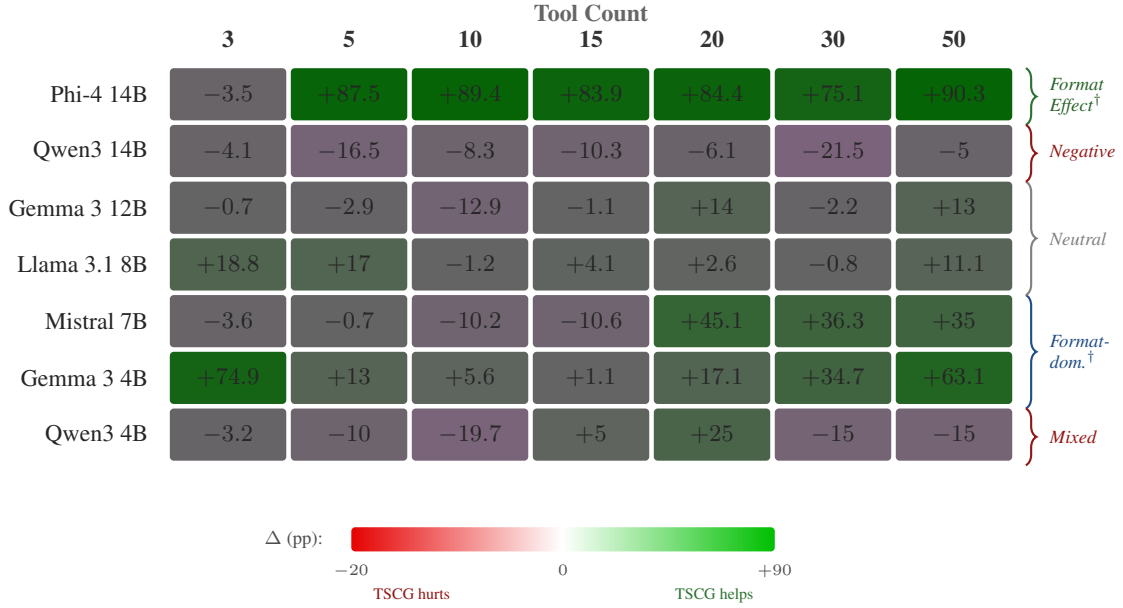


Figure 3: TSCG delta heatmap (Δ = TSCG accuracy – JSON-baseline accuracy, in percentage points) across seven models and seven tool-count conditions. Green cells indicate conditions where TSCG improves over JSON schemas; red cells indicate degradation. Five behavioral clusters emerge: *Format Effect*[†]—Phi-4’s +75–90 pp gains are JSON→text translation, not compression benefit (§6.2); *Format-dominated*[†]—Mistral and Gemma 4B show large deltas driven primarily by format translation; *Neutral*—Llama 3.1 8B and Gemma 3 12B show modest mixed effects; *Negative*—Qwen3 14B shows uniform degradation (−4 to −22 pp); *Mixed*—Qwen3 4B exhibits chaotic response. [†]E1/E4 text-baseline experiments confirm format contribution (§5).

Table 13: Tool-selection accuracy at 20 and 50 tools (Scenario D).

Model	20 Tools		50 Tools		Shift
	json-text	TSCG	json-text	TSCG	
Phi-4 14B	0.0	84.4	0.0	90.3	3→50
Mistral 7B	35.0	80.1	30.0	65.0	20→50
Gemma 3 4B	49.9	67.0	24.3	87.4	15→50
Gemma 3 12B	85.0	95.0	85.0	98.0	—
Llama 3.1 8B	78.4	81.0	75.1	86.3	—
Qwen3 4B	44.3	69.3	90.0	75.0	volatile
Qwen3 14B	90.2	84.1	94.6	89.6	—

Table 14: Format vs. compression decomposition (E4). Format translation dominates; no small model shows genuine compression.

Model	Format Δ	Compr. Δ	Class
Phi-4 14B	+92.0	−7.0	1: Format
Mistral 7B	+44.6	−7.4	1: Format
Gemma 3 4B	+47.3	−8.9	1: Format
Qwen3 4B	+16.7	−23.4	1: Format
Llama 3.1 8B	+5.6	+0.3	3: Neutral
Gemma 3 12B	+9.2	−0.9	3: Neutral

a contribution: empirically characterized, model-specific deployment guidance. The full per-operator heatmap (Figure 5) visualizes the contrast across all tested conditions.

Table 15: Three operator-sensitivity archetypes across frontier models (43-tool MCP catalog, $n=40$ per cell, 2 seeds).

Archetype	Model	Operator Profile
Operator-HUNGRY	Opus 4.7	Every operator helps. CCP alone +20 pp. CFL+CFO synergy +17.5 pp (super-additive). All-8-ops optimal.
Operator-SENSITIVE	GPT-5.2	CFL +2.5 pp, CFO −5 pp, CCP 0 pp. All-8-ops is worst case (−10 pp). CFO must be excluded.
Operator-ROBUST	Sonnet 4	6/7 per-operator conditions identical at 80.0%. Only CFO causes −2.5 pp. Any safe config works.

Table 16: Opus 4.7 scaling: TSCG advantage persists on heavy production schemas despite saturation on light synthetic catalogs.

Scale	Schema Type	json-text	v13-smart	Δ	Savings
43t MCP	light (benchmark)	76.7	80.0	+3.3	56.6%
50t synthetic	light (synthetic)	68.3	76.7	+8.3	55.1%
75t synthetic	light (synthetic)	73.3	73.3	0.0	56.1%
100t synthetic	light (synthetic)	75.0	75.0	0.0	52.5%
43t MCP	heavy (production)	75.0	80.0	+5.0	56.6%

6.4 Opus Scaling: Heavy-Schema Rescue (T1.7)

Five-point Opus 4.7 scaling curve across synthetic and production schemas (Table 16).

TSCG advantage saturates on light synthetic catalogs at 75–100 tools but *persists* on heavy production MCP schemas (+5.0 pp at ~10,500 input to-

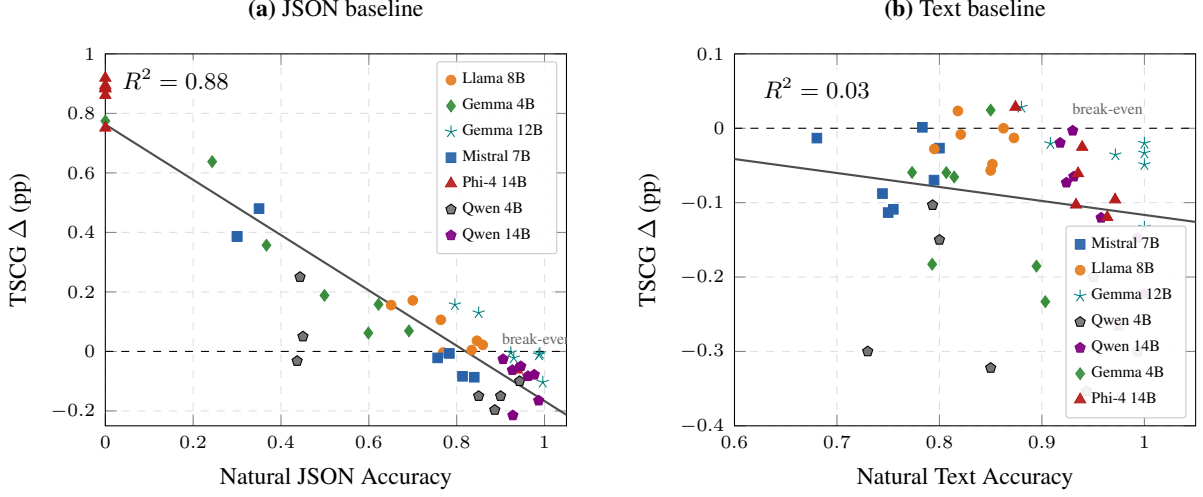


Figure 4: Format-confound decomposition of the predictive model. **(a)** Against JSON baselines ($n=49$, 7 models $\times$ 7 catalog sizes), natural accuracy strongly predicts TSCG improvement ($R^2=0.88$, slope ≈ -0.93): models that fail at JSON parsing gain the most. **(b)** Against text baselines ($n=49$), the correlation vanishes ($R^2=0.03$, $p=0.24$): once the format confound is removed, baseline accuracy has no predictive power. The 97% drop in R^2 confirms that the JSON-baseline regression captures format sensitivity, not compression benefit.

	Opus 4.7	GPT-5.2	Sonnet 4
CFL	+5.0	+2.5	0.0
CFO	+7.5	-5.0	-2.5
CCP	+20.0	0.0	0.0
SAD	+15.0	-2.5	0.0
CFL+CFO	+17.5	-2.5	0.0
all-8	+20.0	-10.0	0.0
	HUNGRY	SENSITIVE	ROBUST

Figure 5: Per-operator isolation heatmap (Δ pp vs. baseline-v10, 43-tool MCP catalog). Opus 4.7 benefits from every operator (HUNGRY); GPT-5.2 is harmed by CFO and all-8-ops (SENSITIVE); Sonnet 4 is invariant except for CFO (ROBUST). Values from leave-one-in methodology ($n=40$ per cell, 2 seeds).

kens per call, with perfect seed stability: 80/80/80; Figure 6). Token savings remain 52–57% throughout. This indicates saturation is catalog-weight-specific, not a fundamental TSCG limit: heavy per-tool schemas carry more parser-ambiguity that structured compilation resolves.

6.5 Benchmark Generalization: TAB $\rightarrow$ MCP (T1.3)

Sonnet 4 on 43-tool MCP Combined: TSCG delta = -1.6 pp on synthetic TAB vs. -1.7 pp on real MCP schemas—the synthetic benchmark predicts real MCP behavior within 0.1 pp. This directly ad-

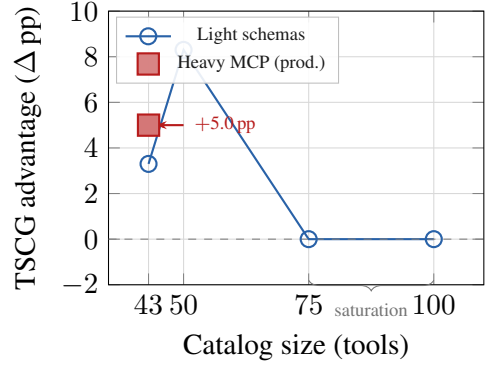


Figure 6: Opus 4.7 TSCG advantage across catalog sizes. Light synthetic schemas saturate at 75–100 tools ($\Delta=0$), but heavy production MCP schemas ($\sim 10,500$ input tokens per call) sustain a $+5.0$ pp advantage at 43 tools with perfect seed stability (80/80/80). Saturation is catalog-weight-specific, not a fundamental TSCG limit.

dresses the “self-constructed benchmark” concern with tight quantitative evidence.

6.6 External Validation

BFCL: 93.2% on Berkeley Function Calling Leaderboard schemas (vs. 85.7% natural; ARR 108%, 46.8% savings)—TSCG *improves* accuracy on third-party benchmarks. Naïve truncation at 16 tools matches TSCG (87.0% vs. 87.8%); at 50 ambiguous tools, TSCG achieves 100% vs. 98.5%, with multi-tool sequencing driving the gap (87.5% vs. 75.0%).

7 Discussion

7.1 Three Mechanisms of Improvement

TSCG’s accuracy improvements arise from three distinct mechanisms. **M1: Format Translation** converts JSON schemas to structured text—the dominant mechanism for Class 1 models, where JSON parsing is the bottleneck (e.g., Phi-4: 0%→84.4% at 20 tools, entirely format-driven per E4 decomposition). **M2: Structural Reorganization** (CAS reordering, CFL constraint positioning, CFO causal ordering) improves accuracy beyond format change—the dominant mechanism for Class 2 frontier models (+10.9 pp mean against native function calling). However, CFL and CFO become counterproductive at ≥ 43 tools, limiting M2’s benefit to CAS at larger catalog sizes. **M3: Token Reduction** decreases attention dilution as a secondary effect. Notably, GPT-4o in Scenario A achieves +1.0 pp accuracy improvement with only 6.2% token savings (Table 1), suggesting M2 (structural reorganization) operates substantially independently of M3. The relative contribution of M2 versus M3 is a direction for future work with controlled ablations.

7.2 Four-Class Behavioral Taxonomy

Two complementary taxonomies emerge from our analysis. The three-archetype classification in Section 6.3 characterizes per-operator response profiles across three frontier models (Opus 4.7, GPT-5.2, Sonnet 4), derived from leave-one-in isolation experiments. The four-class taxonomy below generalizes this analysis to all twelve tested models, combining the archetype data with E4 text-baseline experiments and N3 conservative ablations to capture format-versus-compression behavior across model scales.

E4 text-baseline experiments (2,520 calls), N3 conservative ablation (180 calls), and N1 30B benchmarks (840 calls) yield a four-class taxonomy across 12 models: **Class 1** (Format-dominated: Phi-4, Mistral 7B, Gemma 4B, Qwen3 4B)—large JSON-baseline gains (+17 to +90 pp) that vanish against text baselines (−7 to −23 pp); benefit is purely format translation. **Class 2** (Compression: Claude, GPT-4o, GPT-5.2)—genuine structural compression persists against text baselines (+5 to +11 pp for Claude Sonnet 4). **Class 3** (Neutral: Llama 8B, Gemma 12B, Mistral-Small 24B)—TSCG neither helps nor harms. **Class 4** (Conservative-only: Qwen3 14B, Qwen2.5-Coder

32B)—balanced TSCG degrades accuracy, but conservative SDM improves it (+4.4 pp).

Deployment guidance. Deployment should follow per-model configuration rather than a single universal default. Opus-class models tolerate the full operator pipeline; GPT-class requires operator selection (excluding CFO); Sonnet-class is robust to any safe configuration. For local models: conservative profile for all architectures except Qwen (where native JSON is preferable when available); balanced for frontier APIs. The conservative profile is validated as safe default across 8/9 local models (4B–32B).

Functional validity. A spot-check against the Filesystem MCP server confirms that TSCG-compiled schemas produce executable tool calls on production endpoints, validating that compression preserves call-level correctness beyond benchmark accuracy.

7.3 Why Schema Compression Cannot Be Internalized

A natural question is whether tool-schema compression could be achieved through prompt instructions—asking the model to “internally compress” schemas before reasoning. This approach fails for several architectural reasons.

Tokenization is one-shot and pre-model. The tokenizer operates on the input string before any model computation. By the time the model processes the input, the JSON schema has already been converted to a token sequence. The model cannot retroactively retokenize or remove tokens already in its context window.

Compression instructions add tokens. An instruction to “compress tool schemas internally” is itself additional tokens in the context window. Empirically, attempting this approach increases total token consumption rather than reducing it, while providing no measurable accuracy benefit (we observed this in pilot experiments not included in the main results).

Format effects are pre-attention. As shown by our R^2 decomposition (Section 6.2), 88% of token-cost variance comes from format choice (text vs. JSON), with operators contributing additional refinement. The format choice happens at the input boundary—the model has no mechanism to “re-frame” its inputs mid-stream.

Implications. Our compiler-based approach succeeds specifically because it operates *pre-*

tokenization. This positions schema compression as an architectural choice rather than a prompt-engineering technique, with three practical consequences: (1) compression must happen externally, before the API call; (2) caching of compiled schemas is straightforward (deterministic compilation); (3) the compression layer is model-agnostic—the same compiled output works across any model that accepts text input. This architectural argument explains why our approach is preferable to model-side prompting strategies and motivates our external compilation library design.

7.4 Toward a Community Schema Registry

Our compiler-based approach enables a forward-looking research direction: a community-curated registry of pre-compiled tool schemas. The taxonomy could span:

- **API services:** compiled schemas for Stripe, Slack, Notion, GitHub, and similar widely-used services.
- **MCP servers:** pre-compiled versions of standard Model Context Protocol servers ([Anthropic, 2024](#)) (filesystem, postgres, GitHub MCP, etc.).
- **Database operations:** common SQL, NoSQL, and vector database operations.
- **Cloud providers:** AWS, GCP, Azure operation schemas.
- **Developer tools:** Git, Docker, Kubernetes operations.
- **Productivity tools:** calendar, email, task management schemas.

Each entry would include compiled versions for multiple archetype profiles (conservative, balanced, aggressive), tested-against-models metadata, benchmark results, and version pinning relative to the source API.

Quality assurance considerations. A schema registry requires CI/CD validation that compiled schemas are semantically equivalent to source schemas. This involves: schema-equivalence testing (the compiled schema must invoke the same functions with the same parameters), accuracy benchmarking against the original (to detect regressions when source APIs evolve), and per-model archetype validation (to ensure profile recommendations remain accurate as models update).

Network effects and maintenance. A successful registry would benefit from network effects: as adoption grows, contributors add new schemas, the catalog becomes more comprehensive, and the marginal value of each new contribution increases. However, this creates maintenance obligations: API drift requires re-compilation when source schemas change, and per-version model inversions (Section 6.3) imply profiles need revalidation when major model versions release.

Ecosystem implications. Such a registry could establish schema compression as standard infrastructure rather than an opt-in optimization, analogous to package registries in software ecosystems (npm, PyPI). This represents a qualitatively different contribution from a single tool: it would catalyze ecosystem-level efficiency improvements that no single compression library can achieve in isolation. Future work will explore the technical, organizational, and community-governance challenges involved in building and maintaining such infrastructure.

8 Limitations

Benchmark scope. TAB is self-constructed; we mitigate this with BFCL external validation (ARR 108%) and TAB-to-MCP transfer (0.1 pp delta), but independent third-party evaluation on diverse catalogs is needed.

Task coverage. TAB measures tool selection accuracy and parameter extraction only—not generation quality, multi-turn coherence, or end-to-end task completion.

Small-model degradation. Class 1 models show negative compression gains against text baselines (−7 to −23 pp), meaning TSCG harms accuracy in hypothetical text-mode deployments with <10 tools.

Language. All evaluations and filler patterns are English-only; multilingual extension requires language-specific libraries.

Model coverage. 12 models across 4 architecture families. Generalization to untested architectures (Mamba, RWKV) and thinking models (o1, o3-mini) is uncertain.

Model version evolution. Frontier model APIs are continuously updated; exact performance characterization holds for the tested API versions during March–April 2026. Results may shift as providers deploy model updates.

Predictive model. LOO RMSE of 12.8 pp provides coarse triage; Phi-4’s zero-accuracy leverage points inflate R^2 (excluding Phi-4: $R^2=0.81$).

Scope and choice of approach. Our compiler-based, format-preserving approach achieves moderate compression (50–72%) compared to schema-replacement strategies like MCP Code Mode (Anthropic, 2025; Cloudflare, 2026), which achieve 98–99% reduction. The choice depends on use case: schema replacement excels when all tools have executable code interfaces and a sandboxed runtime is acceptable. Our approach preserves full schema semantics and applies to non-executable tool definitions; we observe this fidelity to be beneficial when models can use parameter constraints during reasoning. Compared to mcp-compressor’s wrapper-tool reduction (Atlassian Labs, 2026) (70–97%), our compression is lower but maintains full schema visibility, which we hypothesize underlies our consistent accuracy improvements (Section 6). A direct head-to-head benchmark against these systems is left to future work.

Evaluation freeze. GPT-5.5 was released after our evaluation period (March–April 2026). Extending TAB to post-freeze models is left to future work.

AI assistance. AI language models assisted with code quality checks, statistical verification, and manuscript editing. All experimental design, data collection, analysis, and scientific conclusions are the sole work of the authors.

9 Conclusion

We have presented Token-Context Semantic Grammar (TSCG), a deterministic tool-schema compiler that addresses the protocol mismatch between JSON-based tool APIs and LLM token processing.

JSON-API enablement. Every production framework transmits tools as JSON, yet small models (4B–14B) achieve only 0–49% accuracy at >15 JSON-format tools. TSCG recovers accuracy to 65–90% by translating JSON schemas to structured text—enabling small models as functional tool-use agents in edge and privacy-constrained environments. E4 decomposition (2,520 Ollama calls across 6 models) identifies format translation as the dominant mechanism for this class.

Frontier compression. For frontier models (Claude Sonnet 4, GPT-4o, GPT-5.2), TSCG provides genuine structural compression (+5–11 pp for Claude Sonnet 4), with 50–72% token savings

that persist when the format confound is eliminated. On heavy production MCP schemas, advantages persist (+5.0 pp) despite saturation on light synthetic catalogs.

Mechanistic decomposition. The first format-versus-compression analysis for tool-schema optimization yields a four-class taxonomy across 12 models (4B–32B + frontier), with concrete deployment recommendations per model class. The predictive model ($R^2=0.88$, collapsing to 0.03 against text baselines) quantifies format sensitivity and enables deployment triage from a single baseline measurement.

TSCG is released as an open-source npm package—1,200 LOC TypeScript, zero dependencies, sub-millisecond execution.³ TAB, the first tool-schema compression benchmark (~19,000 calls, 12 models, 5 scenarios), is released as open-source evaluation infrastructure.

Acknowledgements

This work was conducted independently without institutional funding or affiliation. The author thanks Dominic Wolf⁴ for sharing valuable practical experience with TSCG deployments that informed several design decisions. Portions of the experimental analysis, quality control, and manuscript preparation were assisted by AI language models (Claude, GPT-4), consistent with ACL’s policy on AI writing assistance. All scientific claims, experimental design, data analysis, and final editorial decisions are solely the author’s responsibility.

Data and Code Availability

The TSCG compiler, TAB benchmark suite (task definitions, tool schemas, evaluation harness), and all evaluation scripts are released as open-source software at: <https://github.com/SKZL-AI/tscg>. A persistent archive is available via Zenodo: <https://doi.org/10.5281/zenodo.19795991>. Raw evaluation logs are available on request.

References

Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Open standard

³<https://github.com/SKZL-AI/tscg>

⁴ORCID: <https://orcid.org/0009-0003-5789-1991>

- for connecting AI assistants to data sources. Accessed: 2026-04-28.
- Anthropic. 2025. Code execution with MCP: Building more efficient AI agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>. Engineering blog post. Accessed: 2026-04-28.
- Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. 2024. **LeanContext: Cost-efficient domain-specific question answering using LLMs**. *Natural Language Processing Journal*, 7:100065. ArXiv:2309.00841.
- Atlassian Labs. 2026. mcp-compressor: An MCP server wrapper for reducing tokens consumed by MCP tools. <https://github.com/atlassian-labs/mcp-compressor>. Open-source MCP proxy. Accessed: 2026-04-28.
- Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. **Adapting language models to compress contexts**. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846. Association for Computational Linguistics. ArXiv:2305.14788.
- Joong Ho Choi, Jiayang Zhao, Jeel Shah, Ritvika Sonawane, Mihir Sonawane, Navya Singh, Avani Appalla, Will Flanagan, and Filipe Condessa. 2025. **CompactPrompt: A unified pipeline for prompt and data compression in LLM workflows**. In *2nd Workshop on LLMs and Generative AI for Finance, ACM ICAIF '25*, Singapore. ArXiv:2510.18043. Accessed: 2026-04-28.
- Cloudflare. 2026. Code Mode: Bringing programmatic tool access to AI agents. <https://blog.cloudflare.com/code-mode-mcp/>. Engineering blog post. Accessed: 2026-04-28.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. **Promptbreeder: Self-referential self-improvement via prompt evolution**. *arXiv preprint arXiv:2309.16797*.
- Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujia Yang. 2024. **EvoPrompt: Connecting large language models with evolutionary algorithms yields powerful prompt optimizers**. In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2309.08532.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. **LLMLingua: Compressing prompts for accelerated inference of large language models**. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376. Association for Computational Linguistics. ArXiv:2310.05736.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. **LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression**. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand. Association for Computational Linguistics. ArXiv:2310.06839.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. **DSPy: Compiling declarative language model calls into self-improving pipelines**. *arXiv preprint arXiv:2310.03714*.
- Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. **Compressing context to enhance inference efficiency of large language models**. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353. Association for Computational Linguistics. ArXiv:2310.06201.
- Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. 2025. **Prompt compression for large language models: A survey**. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7182–7195, Albuquerque, New Mexico. Association for Computational Linguistics. ArXiv:2410.12388.
- Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. **Learning to compress prompts with gist tokens**. In *Advances in Neural Information Processing Systems*, volume 36. ArXiv:2304.08467.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. **LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression**. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, Bangkok, Thailand. Association for Computational Linguistics. ArXiv:2403.12968.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. **The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models**. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. **Gorilla: Large language model connected with massive APIs**. In *Advances in Neural Information Processing Systems*, volume 37. ArXiv:2305.15334.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chengguang Zhu, and Michael Zeng. 2023. **Automatic**

- prompt optimization with “gradient descent” and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968. Association for Computational Linguistics. ArXiv:2305.03495.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. Tool-LLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2307.16789.
- Tobias Schnabel and Jennifer Neville. 2024. **Sym-bolic prompt program search: A structure-aware approach to efficient compile-time prompt optimization.** In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 670–686, Miami, Florida. Association for Computational Linguistics. SAMMO framework; arXiv:2404.02319.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. **Neural machine translation of rare words with subword units.** In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725. Association for Computational Linguistics.
- toon-format. 2025. TOON: Token-Oriented Object Notation Specification. <https://github.com/toon-format/toon>. GitHub: toon-format/toon. Accessed: 2026-04-28.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. **Attention is all you need.** In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. ArXiv:1706.03762.
- Xingchen Wan, Ruoxi Sun, Hootan Nakhost, Hanjun Dai, Julian Martin Eisenschlos, Serkan Ö. Arik, and Tomas Pfister. 2023. **Universal self-adaptive prompting.** In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7437–7462. Association for Computational Linguistics. ArXiv:2305.14926.
- Ming Wang, Yuanzhong Liu, Xiaoyu Liang, Songlian Li, Yijie Huang, Xiaoming Zhang, Sijia Shen, Chaofeng Guan, Daling Wang, Shi Feng, Huaiwen Zhang, Yifei Zhang, Minghui Zheng, and Chi Zhang. 2024a. **LangGPT: Rethinking structured reusable prompt design framework for large language models from the programming language perspective.** *arXiv preprint arXiv:2402.16929*.
- Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Hao-tian Luo, Jiayou Zhang, Nebojsa Jojic, Eric P. Xing, and Zhiting Hu. 2024b. **PromptAgent: Strategic planning with language models enables expert-level prompt optimization.** In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2310.16427.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. **Efficient streaming language models with attention sinks.** In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2309.17453.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. **Large language models as optimizers.** In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2309.03409.
- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. **Optimizing generative AI by backpropagating language model feedback.** *Nature*, 639:609–616. Preprint available as arXiv:2406.07496 “TextGrad: Automatic Differentiation via Text”.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. **Large language models are human-level prompt engineers.** In *The Eleventh International Conference on Learning Representations (ICLR)*. ArXiv:2211.01910.

A Pre-TAB Evaluation

The pre-TAB evaluation comprised 19 core tasks across 7 categories (Factual $n=4$, Reasoning $n=4$, Classification $n=2$, Extraction $n=1$, OptFirst $n=3$, Complex $n=2$, NearDup $n=3$), 6 compression strategies, 11 independent runs on Claude Sonnet 4, and domain-specific benchmarks across 4 phases. This evaluation established TSCG’s baseline effectiveness before the multi-model TAB benchmark. Full results are available in the supplementary materials.

B GSM8K-Under-Load

Tool-schema overhead does not degrade reasoning: Claude Sonnet 4 maintains $\sim 81\%$ GSM8K accuracy across 0–50 irrelevant tool definitions (~ 400 calls across both models). TSCG provides a consistent ~ 4.5 pp advantage over natural schemas for Claude Sonnet 4, suggesting that reduced schema overhead frees attention capacity for mathematical reasoning. The effect is statistically significant ($+4.5$ pp, $p = 0.020$) but practically small (Cohen’s $d = 0.11$). GPT-5.2 shows slightly more degradation under schema load but maintains $>75\%$ accuracy throughout.

Table 17: R^2 robustness checks for the accuracy-gap regression.

Configuration	n	R^2	95% CI	p
7-model full set	49	0.885	[0.78, 0.93]	$< 10^{-20}$
7-model sans Phi-4	42	0.813	[0.58, 0.94]	$< 10^{-10}$
Model-level (7 means)	7	0.951	[0.62, 1.00]	< 0.001
Text baseline (E4)	49	0.028	—	0.24

C SDM Ablation Detail

Conservative SDM (filler removal only) is compared with Balanced (full structural compression) on Qwen3-14B (180 calls, N3 corrected re-run at sizes 10/20/50):

- Balanced TSCG degrades accuracy by -6.6 pp mean (90.2% $\rightarrow$ 84.1% at 20 tools, 94.6% $\rightarrow$ 89.6% at 50 tools).
- Conservative SDM *improves* accuracy by $+4.4$ pp mean (90.2% $\rightarrow$ 99.3% at 20 tools, 94.6% $\rightarrow$ 95.0% at 50 tools).
- CAS reordering and bracket elision—not SDM filler removal—trigger the degradation.

Practical recommendation: Conservative SDM for all models < 10 B and all Qwen architectures (including Qwen2.5-Coder 32B, confirmed by N1); Balanced for frontier models where genuine compression persists.

D Predictive Model and LOO Cross-Validation

The accuracy gap $\Delta(m)$ between Natural and TSCG for model m can be predicted from Natural-baseline performance alone:

$$\Delta(m) = \alpha \cdot \text{Natural}(m) + \beta + \epsilon \quad (3)$$

Table 17 reports robustness checks on this regression.

Phi-4 14B is the strongest leverage point ($\Delta R^2 = -0.07$ when removed), but R^2 remains above 0.80 without it. The linear relationship holds at the model level ($R^2 = 0.95$). LOO cross-validation (holding out one model, refitting on remaining six) yields RMSE = 12.8 pp—adequate for deployment triage but insufficient for precise predictions. Qwen3 4B is the only model where the prediction direction is wrong (predicted positive, actual negative), reflecting its volatile behavior.

E Theorem Proofs

Proof of Theorem 3.1. Each token-reducing operator $T_i \in \mathcal{T}_R$ acts on a disjoint subset of tokens:

Table 18: TSCG optimization profiles as implemented in @tscg/core v1.4.0.

Profile	Active Principles
conservative	SDM only. Safe default for all local models (4B–32B).
balanced	SDM, CAS, CFO, DRO, TAS, CCP. CFL/CFO auto-disabled at ≥ 30 tools in the released implementation.
aggressive	All 8 on Claude; 6 on non-Claude (CFL and SAD-F auto-disabled by echo-back guard).
auto	Adaptive: conservative at ≤ 20 tools, balanced (sans CFL/CFO) at 21–40, conservative at > 40 .

- **SDM**: filler patterns (104+ curated, non-overlapping with delimiters/tokenizer targets).
- **DRO**: verbose delimiter phrases (structural role markers).
- **TAS**: suboptimal tokenizer splits (multi-token $\rightarrow$ single-token Unicode).
- **CFL**: positional relocation of output constraint to position 0.

Disjoint action $\implies$ additive composition. $\mathcal{T}_S$ preserves count; $B=0$ excludes expansion:

$$|\text{tok}(\Pi(S))| \leq |\text{tok}(S)| \cdot \left(1 - \sum_{T_i \in \mathcal{T}_R} r_i \cdot f_i(S)\right).$$

□

F Implementation and Deployment

TSCG is implemented as $\sim 1,200$ lines of TypeScript with zero external dependencies. Three deployment modes: CLI (npm), Chrome Extension (Manifest V3), and Web Application (React + Vite). Production bundle: 34.7 KB (11.7 KB gzipped). Sub-millisecond execution for prompts under 4,096 tokens. Four optimization profiles (Table 18):

G Extended Prior-Art Comparison

H Benchmark Configurations and Statistical Detail

Table 20 documents configuration parameters for all benchmark runs.

H.1 Holm-Bonferroni Correction

107 pairwise McNemar tests, family-wise $\alpha = 0.05$. 9/107 achieve significance (Table 21), all from Scenario D with weak JSON baselines.

H.2 Bootstrap Confidence Intervals

I 11-Model Ablation

Opus 4.7. Claude Opus 4.7 emerges as the only operator-HUNGRY archetype in our study: every

System	Det.	Tok.	Caus.	B-Box	Bdgt.	Cmp.	NoTr.	0-Dep
<i>Compression-Based</i>								
LLMLingua (Jiang et al., 2023)	X	X	X	✓	✓	✓	X	X
LLMLingua-2 (Pan et al., 2024)	X	X	X	✓	✓	✓	X	X
Selective Context (Li et al., 2023)	✓	X	X	✓	X	✓	✓	X
LongLLMLingua (Jiang et al., 2024)	X	X	X	✓	✓	✓	X	X
Gist Tokens (Mu et al., 2023)	X	X	X	X	X	✓	X	X
AutoCompressors (Chevalier et al., 2023)	X	X	X	X	X	✓	X	X
LeanContext (Arefeen et al., 2024)	X	X	X	✓	X	✓	X	X
<i>Search-Based</i>								
DSPy (Khattab et al., 2023)	X	X	X	✓	X	X	✓	X
OPRO (Yang et al., 2024)	X	X	X	✓	X	X	✓	X
APE (Zhou et al., 2023)	X	X	X	✓	X	X	✓	X
EvoPrompt (Guo et al., 2024)	X	X	X	✓	X	X	✓	X
PromptBreeder (Fernando et al., 2023)	X	X	X	✓	X	X	✓	X
USP (Wan et al., 2023)	X	X	X	✓	X	X	✓	X
<i>Gradient-Based</i>								
TextGrad (Yuksekgonul et al., 2025)	X	X	X	X	X	X	✓	X
ProTeGi (Pryzant et al., 2023)	X	X	X	X	X	X	✓	X
PromptAgent (Wang et al., 2024b)	X	X	X	✓	X	X	✓	X
<i>Template / Theoretical</i>								
LangGPT (Wang et al., 2024a)	✓	X	X	✓	X	X	✓	X
TSCG (ours)	✓	✓	✓	✓	✓	✓	✓	✓

Table 19: Comparison of 16 prompt optimization systems and TSCG across eight desiderata. **Determ.:** deterministic output for the same input. **Tok-Aware:** optimizes with respect to BPE tokenizer boundaries. **Causal Th.:** grounded in causal attention theory. **Black-Box:** requires no access to model weights or gradients. **Bdgt. Anch.:** budget-constrained anchor duplication. **Compress.:** achieves measurable token compression. **No Train:** requires no model training or fine-tuning. **Zero-Dep:** zero external runtime dependencies. TSCG is the only system satisfying all eight criteria.

Table 20: Experimental configurations.

Parameter	Core/Domain	TAB	Ollama
Temperature	0	0	0
Seed	N/A	42	42
Max tokens	4096	1024	1024
Mode	Text + FC	Text-mode	Text-mode
SDM profile	full	balanced	balanced
API provider	Anthropic/OpenAI	Anthropic	Ollama (local)
Eval. method	Automated	TABEvaluator	TABEvaluator

Table 22: Bootstrap 95% CIs for key deltas (1,000 re-samples, seed=42).

Scenario	Model	Δpp	95% CI	p
Sc. A (16)	Claude S. 4	+10.0	[0.6, 18.3]	0.200
Sc. A (16)	GPT-5.2	+29.7	[16.1, 42.2]	0.004
Sc. B (43)	Claude S. 4	+5.3	[-0.3, 11.3]	0.063
Sc. D (50)	Phi-4 14B	+90.3	—	<0.001
Sc. D (50)	Gemma 3 4B	+63.1	[45.0, 80.0]	<0.001
GSM8K (50)	Claude S. 4	+4.5	[0.5, 8.5]	0.020

Table 21: Significant comparisons (Holm-Bonferroni, $\alpha=0.05$, 107 tests).

Model	Tools	Δpp	p_{adj}	d
Phi-4 14B	5/10/15	+95.0	0.003	6.11
Phi-4 14B	20	+84.4	0.003	4.21
Phi-4 14B	30	+80.0	0.011	2.81
Phi-4 14B	50	+90.3	0.003	6.11
Gemma 3 4B	3	+85.0	0.007	3.34
Gemma 3 4B	50	+63.1	0.004	1.68
Gemma 3 4B	50 ^c	+65.0	0.004	1.73

for Opus-class models.

operator contributes positively, with CCP alone delivering +20 pp improvement and CFL+CFO synergizing super-additively (+17.5 pp vs. expected +12.5 pp). The full 8-operator pipeline is optimal

Disabled Operator	Δ Acc. (pp)		Δ Tokens		Role
	16 t	43 t	16 t	43 t	
–CAS	–1.4	–2.3	–2	0	Accuracy driver
–SDM	–0.5	–1.8	+1	–28	Schema-dependent
–TAS	–0.9	+0.5	+1	–4	Mixed (scale-sensitive)
–SAD-F	+0.5	–0.5	–9	–20	Negligible
–CCP	± 0.0	± 0.0	–85	–306	No effect (token overhead)
–CFO	–0.5	+1.4	0	0	Hurts at ≥ 43 t
–CFL	± 0.0	+0.9	–6	–6	Hurts at ≥ 43 t
–DRO	+0.5	+0.5	+161	+611	Token engine (95% of savings)

Table 23: Leave-one-out ablation averaged across 11 models (9 local 4B–32B + GPT-4o + GPT-5.2), 20 tasks per condition. Baselines: 65.0% at 16 tools, 61.8% at 43 tools. Δ Acc.: accuracy change when operator is removed (negative = operator helps). Δ Tokens: absolute token change. CAS is the only operator with consistent cross-model accuracy benefit. DRO drives 95% of token savings but has no accuracy effect. CFL and CFO become counterproductive at ≥ 43 tools. Claude Opus 4.7 (12th model, not averaged) confirms the ranking; its unique CCP sensitivity (–5 pp) is discussed in §I.